Il Ciclo di Vita dei Dati

La Dashboard Quiz Python

💻 III Liceo Scientifico Biella - Scienze Applicate

Python Biella Group

🗺️ Dalla Risposta al Grafico

Prima di entrare nel codice, ricordiamo il viaggio che ha fatto ogni vostra risposta:

```
Voi rispondete al quiz
       ↓
Viene scritto un CSV (nome_utente, id_domanda, risposta, tempo)
       ↓
Raccogliamo i CSV di tutti in un unico file (risposte_tutti.csv)
       ↓
Polars legge e unisce i dati
       ↓
Streamlit li mostra nella dashboard
```

Quello che vedrete nel codice è esattamente questo percorso, riga per riga.

📁 I Tre File CSV

La dashboard lavora con **tre file** che avete già incontrato:

File	Cosa contiene
`domande.csv`	Le domande e le 4 possibili risposte
`risposte.csv`	Quale numero di risposta è corretto per ogni domanda
`risposte_tutti.csv`	Le risposte date da **tutti** gli studenti

Ogni file è come una "tabella" separata. La magia sta nel **unirle** per ricavare informazioni utili.

1 — Configurazione e Caricamento

```
st.set_page_config(
    page_title="Risultati Quiz Python",
    page_icon="🎮",
    layout="wide"
)
```

`st.set_page_config` va **sempre messa per prima**, prima di qualsiasi altro comando Streamlit. Imposta il titolo del tab del browser, l'icona e il layout. `layout="wide"` dice a Streamlit di usare tutta la larghezza dello schermo invece di una colonna stretta al centro.

```
df_domande = pl.read_csv("domande.csv")
df_risposte = pl.read_csv("risposte.csv")
df_studenti = pl.read_csv("risposte_tutti.csv")
```

Tre `pl.read_csv` — uno per ciascuno dei nostri file. Il risultato è un DataFrame Polars, la struttura che abbiamo già visto: righe e colonne, come una tabella Excel in memoria.

🔗 2 — Il JOIN: Unire le Tabelle

E' la parte più nuova. Abbiamo i dati degli studenti, ma non sappiamo ancora se le loro risposte sono corrette, perché quella informazione sta in un altro file (`risposte.csv`). Dobbiamo **unire** i due DataFrame.

```
df_con_corrette = df_studenti.join(
    df_risposte,
    on="id_domanda",
    how="left"
)
```

Come funziona `.join()`?

Immaginate due liste:

df_studenti (quello che ha fatto ogni studente):

```
nome_utente | id_domanda | numero_risposta_fornita | tempo_risposta
alice       | 1          | 2                       | 1200
alice       | 2          | 3                       | 800
```

df_risposte (la "chiave" delle soluzioni):

```
id_domanda | numero_risposta_corretta
1          | 2
2          | 3
```

Come funziona `.join()`?

Con `.join(df_risposte, on="id_domanda", ...)` stiamo dicendo: "*Per ogni riga di df_studenti, cerca in df_risposte la riga con lo stesso id_domanda e aggiungila*".

Il risultato è:

```
nome_utente | id_domanda | numero_risposta_fornita | tempo_risposta | numero_risposta_corretta
alice       | 1          | 2                       | 1200           | 2
alice       | 2          | 3                       | 800            | 3
```

`how="left"` significa: *tieni tutte le righe della tabella di sinistra (df_studenti), anche se non trovano una corrispondenza a destra*. Nel nostro caso non serve preoccuparsene, ma è buona pratica.

Aggiungere la colonna "corretta"

```
.with_columns(
    (pl.col("numero_risposta_fornita") == pl.col("numero_risposta_corretta"))
    .alias("corretta")
)
```

`.with_columns()` aggiunge una nuova colonna al DataFrame. In questo caso la colonna si chiama `"corretta"` e contiene `True` se lo studente ha risposto giusto, `False` altrimenti. È come fare un confronto riga per riga tra due colonne.

🏆 3 — Classifica con `.group_by()` e `.agg()`

```
classifica = (
    df_con_corrette
    .group_by("nome_utente")
    .agg([
        pl.col("corretta").sum().alias("risposte_corrette"),
        pl.col("id_domanda").count().alias("domande_risposte"),
        pl.col("tempo_risposta").mean().alias("tempo_medio")
    ])
    ...
)
```

Come funziona `.group_by()` ?

Immaginate di avere tutte le risposte mischiate di tutti gli studenti. `.group_by("nome_utente")` le **raggruppa** per studente, come se metteste i foglietti di alice in un mucchio, quelli di marco in un altro, e così via.

Una volta raggruppati, `.agg()` (aggregazione) dice cosa fare con ciascun gruppo:

- `pl.col("corretta").sum()` → somma i `True` (in Python `True` vale `1`, `False` vale `0`), ottenendo il numero di risposte corrette
- `pl.col("id_domanda").count()` → conta quante domande ha risposto quello studente
- `pl.col("tempo_risposta").mean()` → calcola il tempo medio

Aggiungere la percentuale

```
.with_columns(
    (pl.col("risposte_corrette") / pl.col("domande_risposte") * 100)
    .round(1)
    .alias("percentuale_corrette")
)
```

Divide le risposte corrette per il totale, moltiplica per 100, arrotonda a 1 decimale. `.round(1)` equivale a `round()` di Python, ma applicato all'intera colonna.

Ordinare la classifica

```
.sort(["risposte_corrette", "tempo_medio"], descending=[True, False])
```

Ordina prima per risposte corrette (decrescente: dal più alto al più basso), poi — in caso di parità — per tempo medio (crescente: chi ci ha messo meno è più in alto). Due criteri di ordinamento in una sola riga!

📊 4 — Metriche (Statistiche Generali)

```
num_studenti = df_studenti.select("nome_utente").unique().height
```

`.select("nome_utente")` prende solo quella colonna. `.unique()` elimina i duplicati (ogni studente appare molte volte, una per ogni domanda). `.height` è il numero di righe — cioè il numero di studenti distinti.

```
tempo_medio_globale = df_studenti.select(pl.col("tempo_risposta").mean())[0, 0]
```

Calcola la media di tutta la colonna `tempo_risposta`. `[0, 0]` estrae il valore dalla prima riga, prima colonna del risultato (che è un DataFrame con un solo valore).

```
migliore = classifica.row(0, named=True)
peggiore = classifica.row(-1, named=True)
```

`.row(0, named=True)` prende la prima riga come dizionario Python: `{"nome_utente": "alice", "risposte_corrette": 18, ...}`. `-1` prende l'ultima riga. La classifica è già ordinata, quindi la prima è il migliore e l'ultima il peggiore.

5 — Layout con `st.columns()`

```
col1, col2, col3 = st.columns(3)
with col1:
    st.metric("👥 Studenti partecipanti", num_studenti)
with col2:
    st.metric("❓ Domande totali", num_domande)
with col3:
    st.metric("✅ Risposte corrette (media classe)", f"{percentuale_corrette:.1f}%")
```

`st.columns(3)` crea tre colonne affiancate. Tutto quello che mettiamo dentro `with col1:` appare nella prima colonna, e così via. Senza colonne, le metriche si impilerebbero verticalmente una sotto l'altra.

`st.metric()` mostra un numero grande e visibile — perfetto per i KPI (Key Performance Indicators), cioè i valori più importanti a colpo d'occhio.

📈 6 — Classifica con le Barre

```
max_corrette = classifica["risposte_corrette"].max()

for i, row in enumerate(classifica.iter_rows(named=True)):
    st.write(f"{i+1}° **{row['nome_utente']}** — {row['risposte_corrette']} risposte corrette")
    st.progress(int(row["risposte_corrette"]) / int(max_corrette))
```

`.iter_rows(named=True)` trasforma il DataFrame in una sequenza di dizionari, uno per riga — possiamo così usare un normalissimo `for` di Python.

`enumerate()` aggiunge il contatore `i` (0, 1, 2...) che usiamo per mostrare la posizione in classifica (`i+1` perché partiamo da 1, non da 0).

`st.progress()` vuole un valore tra 0.0 e 1.0. Dividiamo le risposte corrette dello studente per il massimo, così chi ha il massimo mostra una barra piena e gli altri proporzionalmente meno.

L'expander

```
with st.expander("📋 Vedi tabella dettagliata"):
    st.dataframe(...)
```

`st.expander()` crea una sezione nascosta che si apre al click. Utile per non sovraccaricare la pagina con troppe informazioni: le tabelle dettagliate sono lì se si vogliono vedere, ma non ingombrano la vista principale.

7 — Difficoltà delle Domande

```
difficolta = (
    df_con_corrette
    .group_by("id_domanda")
    .agg([
        pl.col("corretta").sum().alias("risposte_corrette"),
        pl.col("corretta").count().alias("totale_risposte")
    ])
    .with_columns(
        (pl.col("risposte_corrette") / pl.col("totale_risposte") * 100)
        .round(1)
        .alias("percentuale_corrette")
    )
    .sort("percentuale_corrette")
)
```

Stesso schema di prima, ma invece di raggruppare per studente, raggruppiamo per domanda.

La domanda con la percentuale più bassa è quella che ha messo in difficoltà più studenti.

Poi uniamo con le domande per avere anche il testo:

```
difficolta_con_testo = difficolta.join(
    df_domande.select(["id_domanda", "domanda"]),
    on="id_domanda",
    how="left"
)
```

Un altro join: stavolta aggiungiamo il testo della domanda, che sta in `df_domande`.

`df_domande.select(["id_domanda", "domanda"])` prende solo le due colonne che ci servono, evitando di portarsi dietro risposta_1, risposta_2, ecc.

Colonna con la barra di progresso

```
st.dataframe(
    difficolta_con_testo.select(["id_domanda", "domanda", "percentuale_corrette"]),
    column_config={
        "percentuale_corrette": st.column_config.ProgressColumn(
            "% Risposte Corrette",
            format="%.1f%%",
            min_value=0,
            max_value=100
        )
    }
)
```

`column_config` permette di personalizzare come Streamlit mostra ogni colonna.

`ProgressColumn` trasforma i numeri in barre visive direttamente dentro la tabella — niente codice aggiuntivo, basta specificarlo nella configurazione.

⏱️ 8 — Il Grafico dei Tempi

```
tempo_per_domanda = (
    df_studenti
    .group_by("id_domanda")
    .agg(pl.col("tempo_risposta").mean().alias("tempo_medio"))
    .sort("id_domanda")
)

st.bar_chart(tempo_per_domanda, x="id_domanda", y="tempo_medio", width="stretch")
```

Raggruppiamo le risposte per domanda e calcoliamo il tempo medio. Poi `st.bar_chart()` fa tutto il lavoro grafico. `width="stretch"` fa sì che il grafico si allarghi su tutta la larghezza disponibile.

9 — Domande e Risposte Corrette

```
for row in domande_con_risposta.iter_rows(named=True):
    numero = row["numero_risposta_corretta"]
    risposta_corretta = row[f"risposta_{numero}"]
    st.markdown(f"**#{row['id_domanda']} — {row['domanda']}**")
    st.success(f"✅ {risposta_corretta}")
    st.divider()
```

`numero = row["numero_risposta_corretta"]` prende il numero della risposta corretta (es. `2`). Poi `row[f"risposta_{numero}"]` costruisce dinamicamente il nome della colonna: se il numero è `2`, cerca `"risposta_2"`. È un trucco elegante per evitare un lungo `if/elif`.

`st.success()` mostra una casella verde — Streamlit ha anche `st.error()` (rosso), `st.warning()` (giallo) e `st.info()` (blu) per diversi livelli di importanza.

`st.divider()` disegna una linea orizzontale di separazione tra una domanda e l'altra.

Schema Riassuntivo del Codice

```
1. Configurazione pagina (set_page_config)
2. Caricamento 3 CSV (read_csv)
3. JOIN: unisci risposte studenti con risposte corrette
4. Aggiungi colonna "corretta" (True/False)
5. GROUP BY studente → calcola classifica
6. Mostra metriche generali (st.metric + st.columns)
7. Mostra classifica con barre (st.progress + for loop)
8. GROUP BY domanda → calcola difficoltà
9. JOIN difficoltà con testo domande
10. Mostra tabella difficoltà (ProgressColumn)
11. GROUP BY domanda → calcola tempi medi
12. Mostra grafico tempi (st.bar_chart)
13. Mostra domande e risposte corrette (st.success)
```

💡 I Concetti Chiave da Ricordare

`.join()` unisce due DataFrame usando una colonna comune come "chiave" — come collegare due tabelle Excel con un CERCA.VERT, ma in una sola riga di codice.

`.group_by().agg()` raggruppa le righe per una categoria e calcola riassunti (somme, medie, conteggi) per ciascun gruppo.

`.with_columns()` aggiunge nuove colonne calcolate a partire da quelle esistenti.

`st.columns()` permette di affiancare elementi invece di impilare tutto verticalmente.

`st.expander()` nasconde contenuto opzionale che l'utente può aprire al click.

`column_config` in `st.dataframe()` personalizza l'aspetto di ogni colonna (barre di progresso, formati numerici, ecc.).

Ogni volta che vedete un `.group_by()`*, chiedetevi:*

- *cosa voglio sapere?*
- *su quale gruppo?*

  Il resto viene da sé...

Grazie per l'attenzione...

*"C'è sempre qualcosa da imparare per migliorarci e crescere...**insieme!**"*